

Document de Conception

Projet Ticketeer

Membres du groupe :

Sarah Badsì • Anyas Meziani • Sarah Abdelli

1. Architecture Générale de l'Application

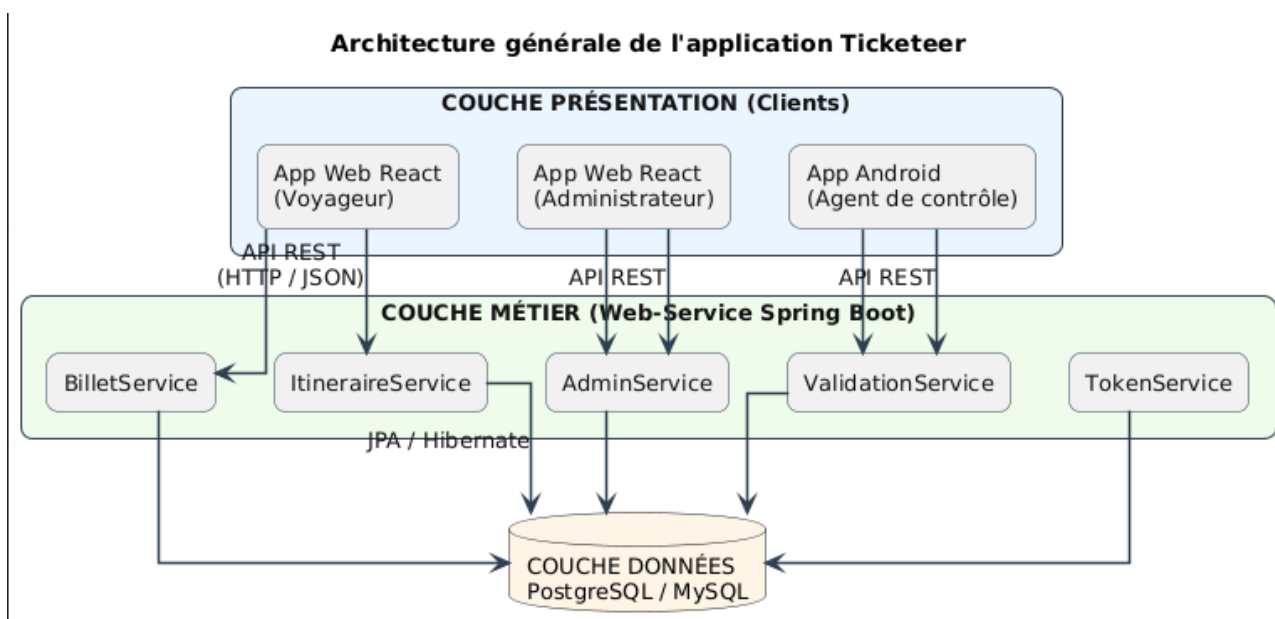
1.1. Vue d'ensemble

L'application Ticketeer repose sur une architecture trois-tiers (3-tier) de type client-serveur, choisie afin d'assurer une séparation claire des responsabilités entre les différentes composantes du système. Cette organisation permet de distinguer de manière structurée la couche présentation, la couche métier et la couche données. Ce choix architectural facilite la maintenance, l'évolutivité, la sécurité ainsi que la réutilisation des composants.

Dans cette architecture, les utilisateurs n'interagissent pas directement avec la base de données. Ils passent par des interfaces dédiées qui communiquent avec un serveur central chargé de traiter les règles métier et d'orchestrer les opérations du système. Le serveur échange ensuite avec la base de données afin de stocker, récupérer et mettre à jour les informations nécessaires au bon fonctionnement de l'application.

L'objectif principal de cette architecture est de garantir une application robuste, modulaire et cohérente. Chaque couche joue un rôle précis : la couche présentation gère l'interaction avec les utilisateurs, la couche métier applique les règles fonctionnelles du projet, et la couche données assure la persistance des informations. Cette séparation permet également de faire évoluer une partie du système sans impacter fortement les autres.

Dans le cadre du projet Ticketeer, cette architecture est particulièrement adaptée, car l'application doit répondre aux besoins de plusieurs profils d'utilisateurs : voyageurs, administrateurs et agents de contrôle. Chacun dispose d'une interface spécifique tout en s'appuyant sur une logique métier centralisée.



1.2. Composants de la couche présentation

Application Web Voyageur (React)

Interface accessible depuis un navigateur web (ordinateur ou mobile). Permet au voyageur de rechercher des itinéraires, sélectionner un trajet, simuler le paiement et afficher son billet numérique avec QR Code.

Application Web Administrateur (React)

Interface d'administration accessible depuis un navigateur web. Permet de gérer le réseau ferroviaire (villes, trains, segments), les comptes clients, et de consulter l'historique des validations pour la traçabilité.

Application Mobile Agent de Contrôle (Android)

Application native Android développée avec Android Studio (Java/Kotlin). Permet à l'agent de s'authentifier, scanner les QR Codes des billets via l'appareil photo, et obtenir un résultat de validation en temps réel (< 2 secondes).

1.3. Composants de la couche métier

Le Web-Service central est développé avec Spring Boot (Java). Il expose une API REST consommée par tous les clients. Il contient les services métier suivants :

Service	Responsabilité
ItineraireService	Recherche de trajets directs et avec correspondance, vérification de compatibilité horaire
BilletService	Création des billets, génération UUID, association aux segments, gestion des états
ValidationService	Vérification token, état du billet, segment/train, date/heure, enregistrement des traces
AdminService	Gestion du réseau (villes, trains, segments), consultation traçabilité
TokenService	Authentification agents, génération et validation des tokens temporaires

1.4. Couche données

La base de données centralisée (PostgreSQL ou MySQL) stocke l'ensemble des entités du système : voyageurs, agents, billets, itinéraires, segments, validations, tokens, villes et trains. Elle garantit la cohérence des données et la traçabilité complète des opérations.

1.5. Communication entre les couches

Toutes les communications entre clients et serveur s'effectuent via des requêtes HTTP/HTTPS utilisant le format JSON. Les échanges sont sécurisés par un mécanisme de tokens JWT pour les agents de contrôle. Les clients React utilisent la bibliothèque Axios pour les appels API. L'application Android utilise Retrofit pour les appels réseau.

Sécurité

Les tokens d'authentification sont temporaires, transmis dans les en-têtes HTTP (Authorization Bearer). Ils ne sont jamais intégrés dans les QR Codes. Toute requête de validation sans token valide est automatiquement rejetée.

2. Outils, Frameworks et Bibliothèques

2.1. Backend — Spring Boot (Java)

Outil / Bibliothèque	Version	Rôle dans le projet
Java (JDK)	17 ou 21 (LTS)	Langage principal du backend
Spring Boot	3.x	Framework principal — API REST, injection de dépendances, configuration automatique
Spring Web (MVC)	inclus Spring Boot	Exposition des endpoints REST (@RestController, @GetMapping, etc.)
Spring Data JPA	inclus Spring Boot	Accès base de données via Hibernate (ORM), repositories automatiques
Spring Security	inclus Spring Boot	Sécurisation des endpoints, gestion des tokens JWT
JWT (jjwt)	0.11.x	Génération et validation des tokens d'authentification des agents
PostgreSQL / MySQL	15+ / 8+	Base de données relationnelle centralisée
Hibernate	inclus JPA	ORM — mapping des entités Java vers les tables SQL
UUID (java.util)	JDK natif	Génération des identifiants uniques des billets
Maven	3.9+	Gestionnaire de dépendances et de build Java
Lombok	1.18+	Réduction du code boilerplate (getters, setters, constructeurs)

2.2. Frontend Web — React

Outil / Bibliothèque	Version	Rôle dans le projet
React	18.x	Framework JavaScript pour l'interface utilisateur (Voyageur + Admin)
Vite	5.x	Outil de build et serveur de développement rapide pour React
React Router	6.x	Navigation entre les pages de l'application (routing côté client)
Axios	1.x	Client HTTP pour les appels vers l'API Spring Boot
Zustand / Redux	4.x	Gestion de l'état global (billet, utilisateur, authentification)

qrcode.react	3.x	Génération et affichage du QR Code à partir de l'UUID du billet
Tailwind CSS	3.x	Framework CSS utilitaire pour le design des interfaces
Node.js / npm	20 LTS	Environnement d'exécution JavaScript et gestionnaire de paquets

2.3. Application Mobile — Android

Outil / Bibliothèque	Version	Rôle dans le projet
Android Studio	Hedgehog+	IDE de développement de l'application mobile de contrôle
Java / Kotlin	Java 11+ / Kotlin 1.9	Langage de développement Android (Java recommandé pour débutants)
Android SDK	API 26+ (Android 8)	Kit de développement Android (niveau API minimal supporté)
Retrofit 2	2.9+	Client HTTP pour les appels vers l'API REST Spring Boot
ZXing (QR Scanner)	3.5+	Bibliothèque de scan et décodage de QR Codes via l'appareil photo
Gson	2.10+	Sérialisation/désérialisation JSON pour les échanges avec l'API
SharedPreferences	natif Android	Stockage local du token d'authentification de l'agent
Gradle	8.x	Système de build Android

2.4. Outils de développement et collaboration

Outil	Rôle
Git / GitHub	Gestion de version du code source et collaboration entre membres du groupe
IntelliJ IDEA	IDE pour le développement Java Spring Boot (Community Edition gratuite)
Postman	Test et documentation des endpoints de l'API REST
DBeaver	Client graphique pour visualiser et gérer la base de données PostgreSQL/MySQL
Docker (optionnel)	Conteneurisation de la base de données et du backend pour faciliter le déploiement

3. Diagramme de Classes de Conception

3.1. Présentation

Le diagramme de classes de conception est une évolution du diagramme d'analyse. Il intègre les détails d'implémentation liés au framework Spring Boot : annotations JPA, relations entre entités, classes de service et contrôleurs REST. Ce diagramme constitue la base directe du code Java à développer.

3.2. Entités JPA (couche données)

@Entity Voyageur

- id: Long (@Id, @GeneratedValue)
- nom: String (@Column, not null)
- email: String (@Column, unique)
- motDePasse: String (@Column)
- billets: List<Billet> (@OneToMany)

@Entity Billet [ENTITÉ CENTRALE]

- uuid: String (@Id) – identifiant unique généré
- dateCreation: LocalDateTime (@Column)
- etat: EtatBillet (@Enumerated, not null)
- voyageur: Voyageur (@ManyToOne)
- itineraire: Itineraire (@ManyToOne)
- validations: List<Validation> (@OneToMany)

+ genererQR(): String
+ estValide(): boolean
+ marquerUtilise(): void
+ invalider(): void

@Entity Itineraire

- id: Long
- segments: List<SegmentTrajet> (@OneToMany, ordered)

+ getVilleDepart(): Ville
+ getVilleArrivee(): Ville
+ estCompatible(): boolean

@Entity SegmentTrajet

- id: Long
- dateDepart: LocalDate
- heureDepart: LocalTime
- heureArrivee: LocalTime
- villeDepart: Ville (@ManyToOne)
- villeArrivee: Ville (@ManyToOne)
- train: Train (@ManyToOne)

+ estDansLaBonnePlage(): boolean
+ estAssocieAuTrain(): boolean

@Entity Validation

- id: Long
- dateHeure: LocalDateTime
- resultat: ResultatValidation
- motifRefus: String (nullable)
- billet: Billet (@ManyToOne)
- agent: AgentControle (@ManyToOne)

@Entity AgentControle

- id: Long
- nom: String
- login: String (@Column, unique)
- motDePasse: String
- tokens: List<Token> (@OneToMany)

+ sAuthentifier(): Token

@Entity Token	@Entity Ville	@Entity Train
- id: Long - valeur: String - dateExpiration: LocalDateTime + estValide(): boolean + estExpire(): boolean	- id: Long - nom: String (unique) + getNom(): String	- id: Long - numero: String (unique) + getNumero(): String

3.3. Énumérations

<<enumeration>> EtatBillet	<<enumeration>> ResultatValidation
VALIDE – billet utilisable UTILISE – billet consommé INVALIDE – billet rejeté	ACCEPTEE – validation réussie REFUSEE – validation échouée

3.4. Services et Contrôleurs REST

Classe	Annotation Spring	Endpoints principaux
ItineraireController	@RestController /api/itineraires	GET /search?depart=&arrivee=
BilletController	@RestController /api/billets	POST /create, GET /{uuid}, GET /{uuid}/qr
ValidationController	@RestController /api/validations	POST /validate, GET /history/{uuid}
AdminController	@RestController /api/admin	POST /villes, POST /trains, POST /segments
AuthController	@RestController /api/auth	POST /login (retourne token JWT)

3.5. Relations entre entités

Entité Source	Relation	Entité Cible	Description
Voyageur	1 — 0..*	Billet	Un voyageur peut avoir plusieurs billets
Billet	* — 1	Itineraire	Chaque billet est associé à un seul itinéraire
Billet	1 — 0..*	Validation	Un billet peut avoir plusieurs validations
Itineraire	1 — 1..*	SegmentTrajet	Un itinéraire contient 1 ou plusieurs segments
SegmentTrajet	* — 1	Ville (départ)	Chaque segment a une ville de départ

SegmentTrajet	* — 1	Ville (arrivée)	Chaque segment a une ville d'arrivée
SegmentTrajet	* — 1	Train	Chaque segment est associé à un train
AgentControle	1 — 0..*	Token	Un agent peut avoir plusieurs tokens (sessions)
AgentControle	1 — 0..*	Validation	Un agent effectue plusieurs validations

